Ingegneria del Software — Soluzione del Tema 18/06/2021

Esercizio 1

Si consideri la seguente classe Java `SensorsData` per salvare e analizzare valori di temperatura provenienti da sensori ambientali. Ogni sensore è rappresentato da un identificativo univoco di tipo `String` e ogni valore di temperatura è rappresentata da un `float`. Per ogni sensore sono salvati solo i 100 valori di temperatura più recenti.

```
public class SensorsData {
  // Ritorna gli identificativi di tutti i sensori per cui e' salvato
  // almeno un valore di temperatura.
  public /*@ pure @*/ Set<String> sensors();

  // Ritorna il numero di valori salvati per il sensore con identificativo id.
  public /*@ pure @*/ int numValuesFor(String id);

  // Ritorna l'i-esimo valore di temperatura salvato per il sensore con identificativo id.
  // I valori sono in ordine di inserimento, dal meno recente (posizione 0) al
  // piu' recente (posizione 99).
  public /*@ pure @*/ float getValue(String id, int i);

  // Aggiunge il valore di temperatura temp per il sensore con identificativo id (aggiungendolo
  // se non presente). Se sono gia' salvati 100 valori, il meno recente viene cancellato.
  public void add(String id, float temp);

  // Ritorna l'insieme di identificativi di tutti i sensori per cui la somma
  // dei valori registrati supera il valore t.
  // Lancia una NegativeThresholdException se il valore di t e' negativo.
  public /*@ pure @*/ Set<String> sensorsExceeding(float t) throws NegativeThresholdException;
}
```

Domanda a)

Si specifichi in JML il metodo `add()`.

Soluzione

Definiamo un predicato JML `unchanged(String s)` come segue:

```
//@ unchanged(String s) = sensors().contains(s) &&
//@     numValuesFor(s)==\old(numValuesFor(s)) &&
//@     (\forall int i; i>=0 && i<numValuesFor(s);
//@         getValue(s, i)==\old(getValue(s, i)))

//@ensures sensors().contains(id) &&
//@ (\forall String s; \old(sensors()).contains(s) && !s.equals(id); unchanged(s)) &&
//@ (\forall String s; sensors().contains(s) && !s.equals(id); \old(sensors()).contains(s)) &&
//@
//@ !\old(sensors()).contains(id) ==> (
//@     numValuesFor(id)==1 && getValue(id, 0)==temp) &&
//@
//@ \old(sensors()).contains(id) ==> ( getValue(id, numValuesFor(id)-1)==temp &&
//@     (\old(numValuesFor(id)) < 100 ==>
//@         numValuesFor(id)==\old(numValuesFor(id))+1 &&
//@         (\forall int i; i>=0 && i<\old(numValuesFor(id)-1);
//@             getValue(id, i)==\old(getValue(id, i)) ) ) &&
//@     (\old(numValuesFor(id)) == 100 ==>
//@         numValuesFor(id)==\old(numValuesFor(id)) &&
//@         (\forall int i; i>=1 && i<\old(numValuesFor(id));
//@             getValue(id, i-1)==\old(getValue(id, i)) ) ) )
public void add(String id, float temp);
```

Domanda b)

Si specifichi in JML il metodo `sensorsExceeding()`.

Soluzione

```
//@ensures t>=0 && (\forall String s; ;
//@    \result.contains(s) <==> ( sensors().contains(s) &&
//@       (\sum int i; i>=0 && i<numValuesFor(id); getValue(s, i)) > t) )
//@signals NegativeThresholdException && t<0
public /*@ pure @*/ Set<String> sensorsExceeding(float t) throws NegativeThresholdException;
```

Domanda c)

Si consideri un'implementazione che utilizza una mappa per contenere i valori di temperatura, come mostrato di seguito.

```
public class SensorsData {
  private final Map<String, List<Float>> values;
  ... }
```

Per tale implementazione, si definiscano l'invariante di rappresentazione e la funzione di astrazione.

Soluzione

Nell'invariante di rappresentazione escludiamo che la mappa sia nulla o vuota, che contenga liste nulle, liste vuote o liste con più di 100 elementi.

```
//@(* Representation invariant RI *)
//@private invariant values != null && values.size() > 0 &&
//@ (\forall String s; values.containsKey(s);
//@    values.get(s)!=null && !values.get(s).contains(null) &&
//@    values.get(s).size()>0 && values.get(s).size()<=100 )
```

La funzione di astrazione definisce `sensors()`, `numValuesFor()` e `getValue()` in funzione della mappa usata nella rappresentazione. Gli altri metodi pubblici sono infatti definiti a partire da essi.

```
//@(* Abstraction function AF *)
//@private invariant
//@ (\forall String s; ; sensors().contains(s) <==> values.containsKey(s)) &&
//@ (\forall String s; values.containsKey(s);
//@     numValuesFor(s) == values.get(s).size() &&
//@     (\forall int i; i>=0 && i<numValuesFor(s); getValue(s, i)==values.get(s).get(i))
```

Domanda d)

Si consideri ora una classe `SensorsData2` che aggiunge un metodo `addSensor(String id)` che aggiunge un sensore (senza alcun valore di temperatura associato). È possibile definire `SensorData2` come sottoclasse di `SensorData` in accordo con il principio di sostituzione?

Soluzione

Non è possibile, in quanto l'aggiunta del metodo porterebbe a violare una proprietà (invariante pubblico) della classe, ovvero che essa include solo sensori per cui è salvato almeno un valore di temperatura.

Esercizio 2

Si consideri la seguente classe Java:

```
public class GPSTrack {
  private double latitude[], longitude[];
  private int numPoints;
  public GPSTrack() {
    latitude = new double[1000];
    longitude = new double[1000];
    numPoints = 0;
  }
  public void addPoint(double latitude, double longitude) {
    synchronized(this.latitude) {
      this.latitude[numPoints] = latitude;
      this.longitude[numPoints] = longitude;
      numPoints++;
    }
  }
  public double getLatitude(int point) {
    synchronized(this.latitude) {
      return this.latitude[point];
    }
  }
  public double getLongitude(int point) {
    synchronized(this.latitude) {
      return this.longitude[point];
    }
  }
}
```

Domanda a)

La classe è correttamente sincronizzata? In caso affermativo spiegare brevemente perchè. In caso negativo spiegare dove sia il problema e mostrare come vada cambiato il codice per ottenere una corretta sincronizzazione.

Soluzione

La classe è correttamente sincronizzata. Tutti i metodi che modificano lo stato dell'oggetto acquisiscono il lock sull'oggetto `this.latitude` il cui riferimento non cambia per tutta la vita dell'istanza.

Domanda b)

Si aggiunga alla classe `GPSTrack` un metodo `public void wait100()` che sospende il chiamante fin tanto che non siano stati aggiunti almeno 100 punti alla traccia. Si chiarisca se e come vadano cambiati gli altri metodi.

Soluzione

```
  public void wait100() throws InterruptedException {
    synchronized(this.latitude) {
      while(numPoints<100) this.latitude.wait();
      return;
    }
  }
```

Bisogna anche aggiungere l'istruzione `this.latitude.notifyAll()` all'interno della parte sincronizzata del metodo `addPoint`.

Domanda c)

Supponendo l'esistenza di una classe `Printer` così fatta:

```
class Printer {
  static void plot(double lat, double lon) { ... }
}
```

il cui metodo `plot` stampa su una mappa il punto di coordinate indicate, si aggiunga alla classe `GPSTrack` un metodo: `public void plot()` che, **in parallelo rispetto al chiamante**, stampa tutti i punti della traccia.

Soluzione

```
  public void plot() {
    double lat[], lon[];
    int np;
    synchronized(this.latitude) {
      lat = new double[1000];
      lon = new double[1000];
      np = numPoints;
      for(int i=0; i<numPoints; i++) {
        lat[i] = this.latitude[i];
        lon[i] = this.longitude[i];
      }
    }
    new Thread( () -> {
        for(int i=0; i<np; i++) Printer.plot(lat[i], lon[i]);
    }).start();
  }
}
```

Esercizio 3

Si consideri questo frammento di codice Java:

```
public class Libro{
   public void leggi(){
      System.out.println("leggo tutto");
   }

   public void leggi(int pag){
      System.out.println("leggo pagina "+ pag);
   }
}

public class LibroConAudio extends Libro{
   public void ascolta(){
      System.out.println("ascolto");
   }

   public void leggi(){
      System.out.println("leggo e ascolto");
   }
}
```

```
1  Libro book  = new Libro();
2  LibroConAudio book2 = new LibroConAudio();
3  Libro sound = new LibroConAudio();
4  LibroConAudio sound2 = new Libro();
5  book.leggi();
6  sound.leggi();
7  book2.leggi();
8  sound2.leggi();
9  sound.leggi();
10 book.leggi(2);
11 book2.leggi(2);
12 sound.leggi(2);
13 sound2.leggi(2);
```

Domanda a)

Scrivere il numero di riga delle istruzioni (se ne esistono) che generano un errore in compilazione.

Soluzione

Riga 4, in quanto `Libro` non è sottoclasse di `LibroConAudio`. Di conseguenza anche le righe 8 e 13.

Domanda b)

Supponendo di eliminare le eventuali righe che generano errori, si scriva, per ogni riga numerata, il valore stampato in uscita.

Soluzione

5: leggo tutto
6: leggo e ascolto
7: leggo e ascolto
9: leggo e ascolto
10: leggo pagina 2
11: leggo pagina 2
12: leggo pagina 2

Esercizio 4

Si consideri il seguente metodo Java:

```
public static int test(int x, int[] a) {
   int count;
   if (x <= 0 || a == null) return 0;
   count = x;
   while (count > 1) {
     if (count > 0) count--;
     else return count;
   }
   return count;
}
```

Domanda:

Per ciascuno dei seguenti criteri di copertura, determinare un insieme minimale di casi di test che massimizzi la copertura:

1. Copertura delle istruzioni;
2. Copertura delle decisioni (branch);
3. Copertura delle decisioni e delle condizioni.

Soluzione

Copertura delle istruzioni. Non si può ottenere una copertura totale, in quanto non si può mai entrare nel ramo `else` del secondo `if` (quello all'interno del ciclo), che risulta vero in tutti i casi in cui si entra nel ciclo medesimo.

- (x=2, a=null) copre il ramo `if`.
- (x=2, a=[1]) entra nel `while` una volta (eseguendo il ramo `if` e decrementando `count`) e infine esegue l'ultima istruzione `return`

Copertura delle decisioni (branch). I casi di test riportati sopra massimizzano anche il criterio di copertura delle decisioni.

- Il primo caso valuta positivamente il primo `if`.

- Il secondo caso valuta negativamente il primo `if`, positivamente e negativamente il `while`, positivamente il secondo `if`.

Copertura delle decisioni e delle condizioni. Per massimizzare la copertura, oltre ai casi precedenti occorre aggiungere un caso che renda falsa la prima condizione della prima decisione (primo `if`).

- (x=0, a=qualsiasi) per valutare positivamente la prima condizione del primo `if`.